

Love Letter
Projet AP4B
Version de pré-rendu

Julie BENED
Corentin HAUTEFAYE
Loïc MORIN
Théo RIGAL

19 décembre 2025

Introduction

Dans le cadre de l'UE AP4B (*Semestre A25*), nous avons réalisé une adaptation du jeu *Love Letter* en Java. L'objectif principal de ce projet est de concevoir une application fonctionnelle, robuste, clairement structurée, tout en respectant les principes fondamentaux de la programmation orientée objet.

L'accent est ainsi mis sur la conception UML, l'organisation du code ainsi que la capacité à modéliser efficacement les entités essentielles d'un moteur de jeu simple.

Par ailleurs, cette adaptation vise à préserver l'esprit et les règles du jeu original, tout en y intégrant une dimension créative et ludique en lien avec le contexte universitaire.

Table des matières

1	Présentation	4
1.1	Règles du jeu	4
1.2	Conventions	5
1.3	Outils utilisés	5
2	Réalisation du jeu	6
2.1	Choix	6
2.2	Architecture générale	6
2.3	Organisation des paquets	6
3	Conception UML	7
3.1	Diagramme de classes	8
3.2	Diagramme de cas d'utilisation	9
3.3	Diagramme de séquence	10
4	Retour d'expérience	11
4.1	Organisation	11
4.2	Difficultés rencontrées	11
4.3	Suggestions d'amélioration	11
5	Conclusion	12

1 Présentation

1.1 Règles du jeu

Love Letter est un jeu de cartes qui se joue de 2 à 6 joueurs et dont le principe est assez simple. Vous incarnez un courtisan qui cherche à transmettre une missive d'amour à la princesse, tout en éliminant ses opposants.

Mise en place Vous disposez d'un ensemble de 21 cartes personnages mélangées, face cachée ; il s'agit de la pioche. La première carte en est defauscée et est placée **face cachée** dans la pile de jeu. Si la partie comporte exactement deux joueurs, alors il convient d'enlever trois cartes supplémentaires de la pioche ; ces dernières sont placées dans la pile, **face visible**. Ensuite, une carte est distribuée à chaque joueur ; cela constitue leur main. Le premier joueur à commencer est celui qui a remporté la dernière manche. S'il s'agit de la première manche, ou qu'il y a eu égalité, il est choisi aléatoirement.

Victoire d'une manche Une manche arrive à échéance lorsqu'une des deux conditions suivantes est remplie :

- Être le dernier joueur en partie
- La pioche est vide

Dans le second cas, une fois que le tour du joueur en cour est terminé, chacun dévoile sa main. Celui qui dispose de la plus grande carte remporte la manche. Notons que les égalités sont tolérées. Enfin à l'issue de la manche, chaque vainqueur gagne un point faveur.

Victoire d'une partie On considère qu'il y a un total de treize points faveur par partie. Ainsi, elle est gagnée s'il existe au moins un joueur dont le score est d'au moins la valeur indiquée dans le tableau ci-dessous :

Nombre de joueurs	2	3	4	5	6
Score requis	6	5	4	3	3

Il est de plus à noter qu'il est toujours possible de gagner une partie avec le nombre de pions faveur de départ, en respectant ce tableau.

Tour des joueurs Le jeu s'effectue dans le sens horaire. Chaque joueur procède en trois étapes :

1. Piocher une carte du paquet
2. Choisir une de ses deux cartes
3. Jouer la carte, résoudre son effet et la place **face visible** dans la pile

Quitter la manche Lorsque le joueur est contraint de quitter la manche, il doit défausser sa main **face visible** et passe bien sûr son tour à chaque itération.

Cartes personnages Les cartes en question ont été adaptées à un contexte UTBM et sont décrites en section 2.1.

1.2 Conventions

Afin d’assurer une meilleure lisibilité ainsi qu’une cohérence globale du projet, plusieurs conventions ont été adoptées tout au long du développement.

Tout d’abord, le code source et les commentaires sont rédigés en Anglais ; cela est en effet conforme aux standards couramment utilisés en programmation, notamment en entreprise. La typographie utilisée est la norme Java : les noms de classes utilisent la notation *PascalCase*, tandis que les noms de méthodes et attributs suivent le format *camelCase*. Les constantes sont placées dans un fichier dédié (`Const.java`) et sont notées en majuscules avec des *underscores* en tant que séparateurs. En sus de cela, les classes spéciales comme les énumérateurs et interfaces, sont respectivement préfixées par un **E** et un **I**. Enfin, les commentaires suivent le format *Doxygen* ; ainsi la signature de chaque fonction/méthode est donnée en entête du corps.

Les diagrammes UML respectent les conventions classiques de modélisation. Les attributs, méthodes et relations sont clairement identifiées. Notons toutefois que les méthodes triviales, comme les *accesseurs* et *mutateurs*, ne sont pas représentés.

1.3 Outils utilisés

Pour réaliser ce projet, les outils suivants ont été utilisés :

- **IntelliJ IDEA** pour l’environnement de développement
- **Git** et **GitHub** pour le contrôle des versions
- **L^AT_EX** pour la rédaction du rapport
- **tikz-uml** pour dessiner les diagrammes UML en L^AT_EX

Aucune version de Java n’a été spécifiée pour ce sujet. Ainsi, nous avons choisi d’utiliser Java 21. En effet, il s’agit d’une version **LTS** (*Long Term Support*), i.e une version stable pour le développement, et qui reste relativement moderne et sécurisée.

2 Réalisation du jeu

2.1 Choix

ADAPTATION UTBM ICI

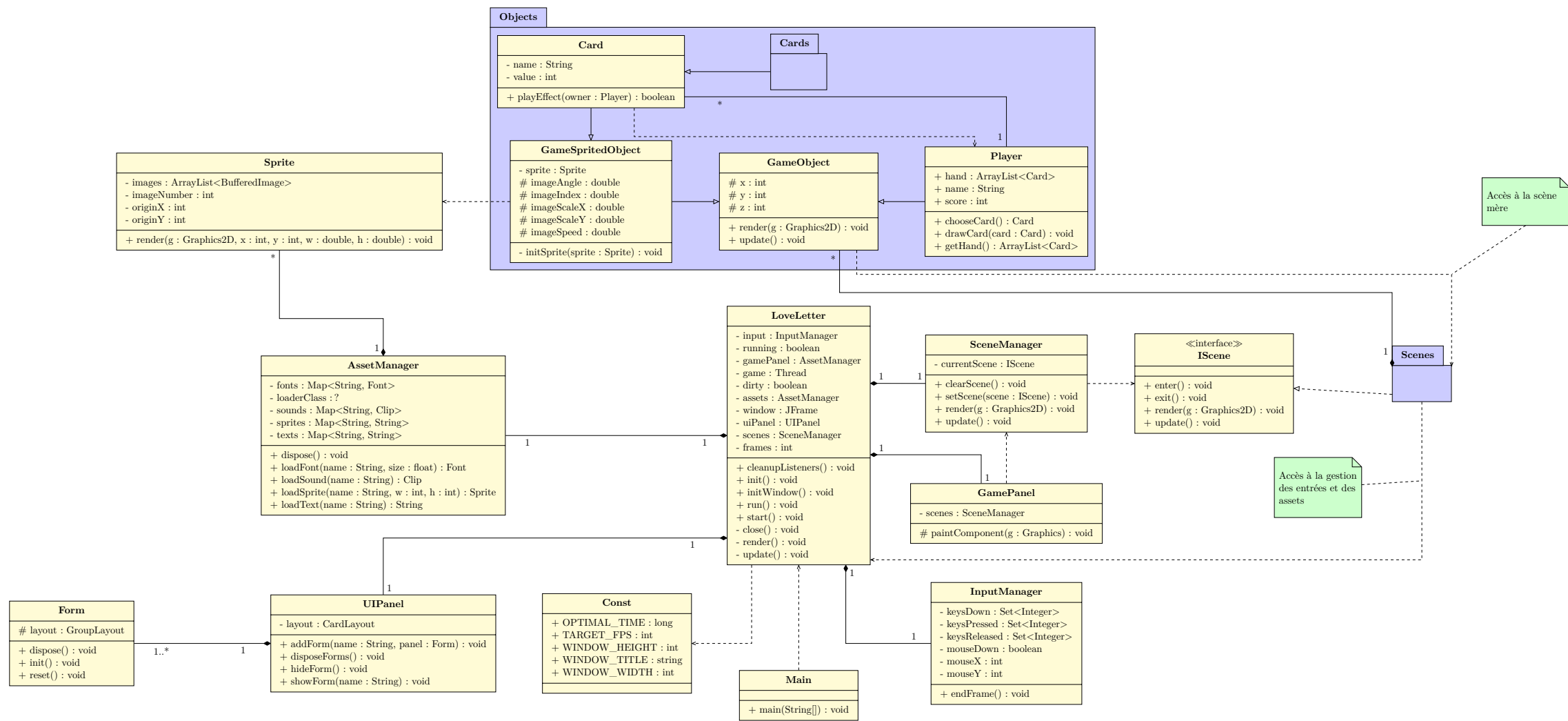
2.2 Architecture générale

2.3 Organisation des paquets

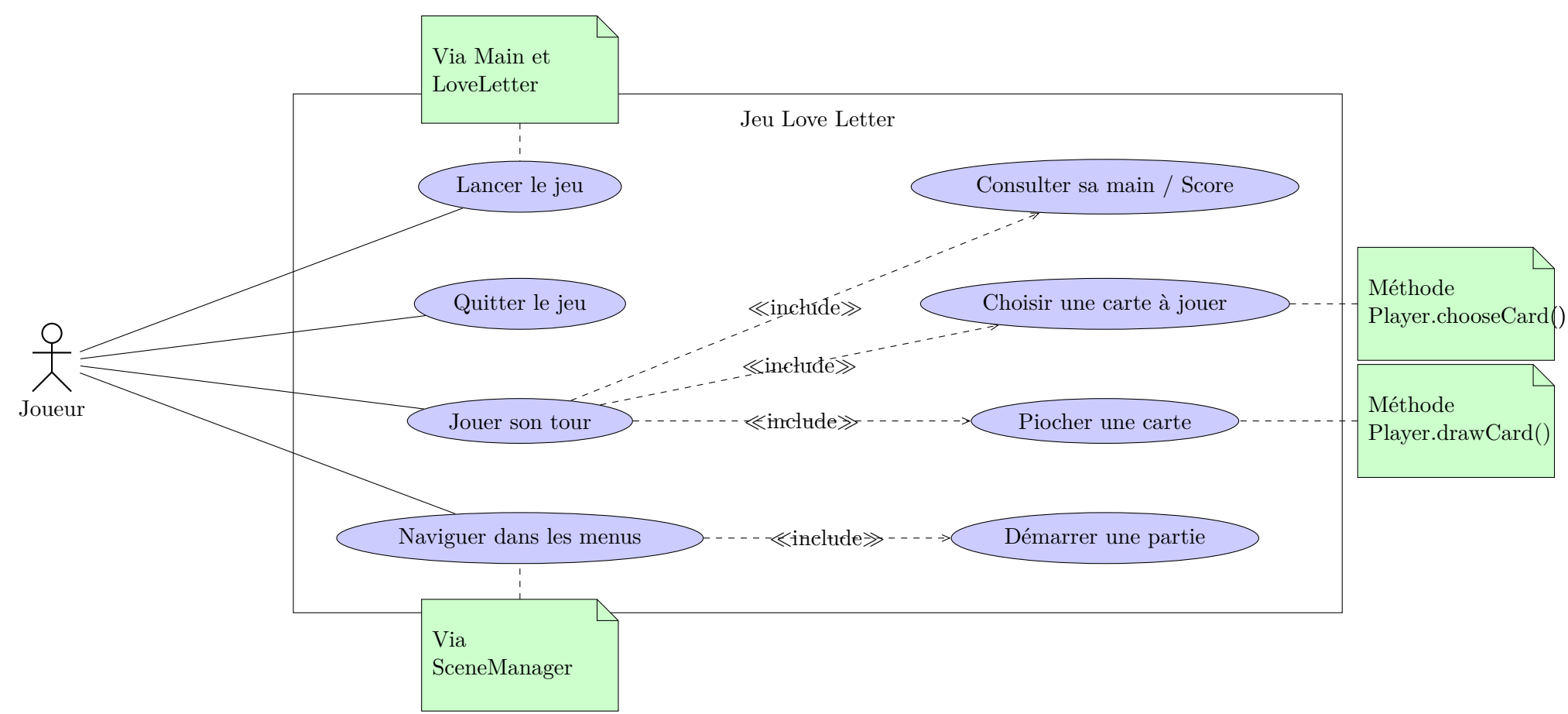
3 Conception UML

EXPLIQUER ICI LES LIAISONS DE UML + DISCLAIMER

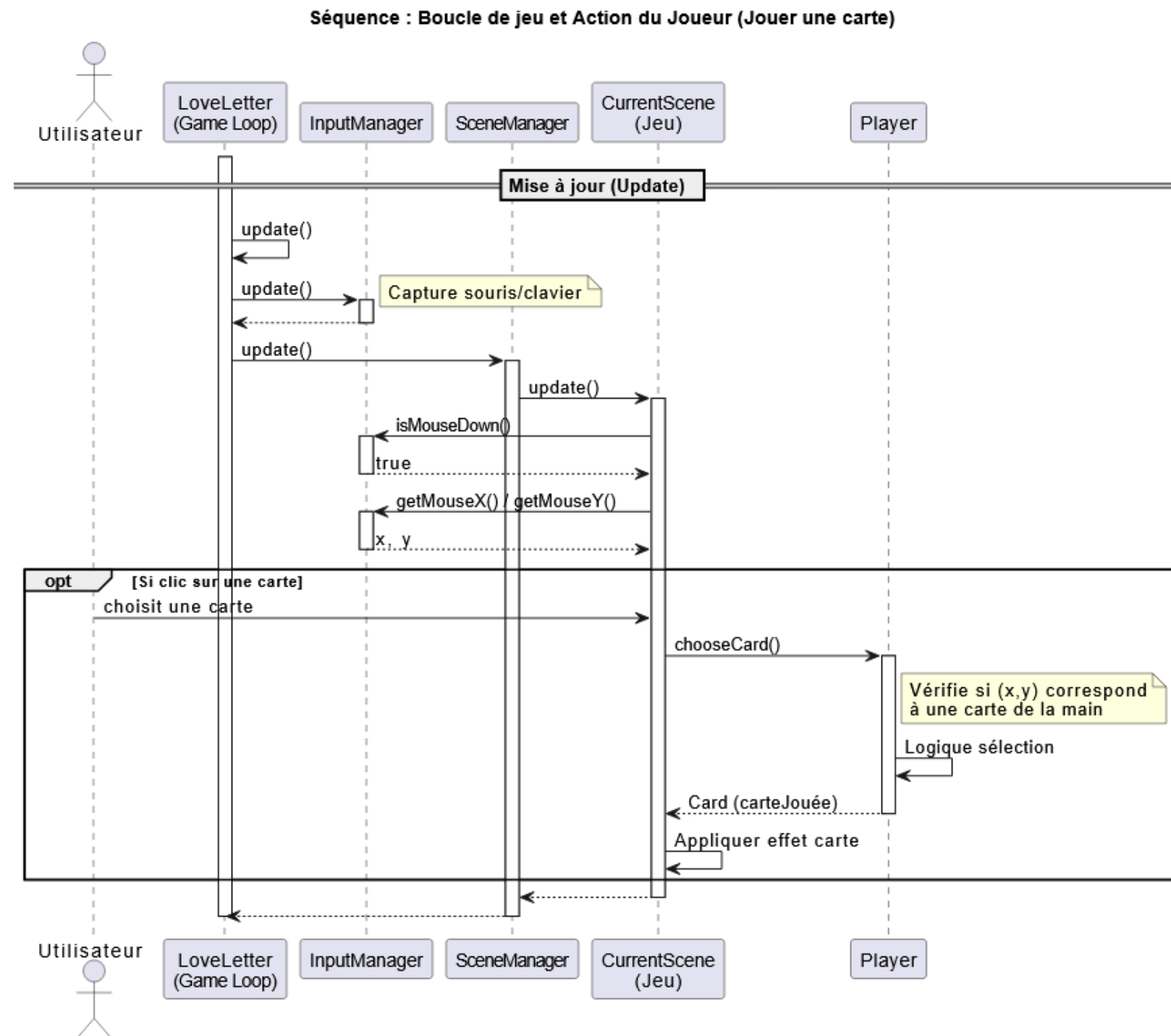
3.1 Diagramme de classes



3.2 Diagramme de cas d'utilisation



3.3 Diagramme de séquence



4 Retour d'expérience

4.1 Organisation

4.2 Difficultés rencontrées

4.3 Suggestions d'amélioration

5 Conclusion